

Danmarks
Tekniske
Universitet



Software Engineering Project Report 2

02161 Software Engineering I
DTU Compute, Spring 2026

Group 16

AUTHORS

Nicolai Søgaard -s234146
Nat Skovlund - s245203
Mathias Kjølbye - s240803
Vedanta Sinha - s245010
Viktor Munk - 240335

Git Repository:

<https://gitlab.gbar.dtu.dk/XXXX/software-project>
7. maj 2026

Indhold

1	Introduction	1
1.1	Scope	1
1.2	Relation to Report 1	1
2	Implemented System Overview (Vedanta Sinha)	2
2.1	Implemented Functionality	2
2.2	Architecture Overview	2
2.3	Traceability to Report 1	2
3	Systematic Tests (White-Box Tests)	3
3.1	Choice of Methods	3
3.2	ProjectService.createProject(...)(Vedanta)	3
3.2.1	Method	3
3.2.2	Input Properties	4
3.2.3	Concrete Test Cases	4
3.2.4	Implementation of Tests	4
3.3	ActivityService.setActivityDetails(...)(Viktor)	4
3.3.1	Method	4
3.3.2	Input Properties	5
3.3.3	Concrete Test Cases	6
3.3.4	Implementation of Tests	6
3.4	ActivityService.addDeveloperToActivity(...)(Viktor)	6
3.4.1	Method	6
3.4.2	Input Properties	7
3.4.3	Concrete Test Cases	8
3.4.4	Implementation of Tests	8
3.5	TimeRegistrationService.registerTime(...)(Nat)	8
3.5.1	Method	8
3.5.2	Input Properties	10
3.5.3	Concrete Test Cases	10
3.6	AbsenceService.registerAbsence(...)	10
3.6.1	Method	10
3.6.2	Input Properties	10
3.6.3	Concrete Test Cases	11
3.7	Summary of White-Box Testing	11
4	Design by Contract ()	12
4.1	ProjectService.createProject(...)(Vedanta Sinha)	12
4.1.1	Method with Assertions	12
4.1.2	Contract	12
4.1.3	Discussion	12
4.2	ActivityService.setActivityDetails(...)(Viktor)	12

4.2.1	Method with Assertions	12
4.2.2	Contract	14
4.2.3	Discussion	14
4.3	<code>ActivityService.addDeveloperToActivity(...)</code> (Viktor)	14
4.3.1	Method with Assertions	14
4.3.2	Contract	15
4.3.3	Discussion	15
4.4	<code>TimeRegistrationService.registerTime(Nat)</code>	16
4.4.1	Method with Assertions	16
4.4.2	Contract	18
4.5	Discussion	18
4.6	<code>AbsenceService.registerAbsence(...)</code>	18
4.6.1	Method with Assertions	18
4.6.2	Contract	19
4.7	Discussion of Contracts	19
5	Code Coverage (Viktor)	20
5.1	Measurement Setup	20
5.2	Results	20
5.3	Analysis by Layer	21
5.3.1	Service layer 77.25%	22
5.3.2	Domain layer 79.77%	23
5.4	Effect of the White-Box Tests	24
5.5	Limitations	25
5.6	Summary	25
6	Design Patterns and SOLID Principles (Nicolai)	26
6.1	Overall Design Choices	26
6.1.1	Domain Layer	27
6.1.2	Repository Layer	27
6.1.3	Service layer	28
6.1.4	User Interface Layer	29
6.1.5	Overall Design Considerations	29
6.2	SOLID-principles	29
6.3	Single Responsible Principle	30
6.3.1	Domain-classes	30
6.3.2	Repositories	30
6.3.3	Service layer	31
6.4	Dependency Inversion Principle	31
6.5	Other SOLID-Principles	33
6.5.1	OCP, Open/Close principle	33
6.5.2	Interface Segregation Principle	33
6.5.3	Liskov Substitution Principle	34
6.5.4	Final remarks	34

7 Conclusion	35
7.1 Reflection on the Development Process	35
7.2 Collaboration and Contribution Weights	35
8 Contribution Table	36
9 Appendix	37
9.1 Git Repository	37
9.2 Run Instructions	37
List of Figures	38
List of Tables	39
References	40

1 Introduction

This report shows the implementation and testing of the project management system, which allows admins to create projects and activities for the former, allocate developers to projects, use a project's allocated time to track progress, and generate reports to keep teams, management, and stakeholders aligned.

The main objectives of this report are to systematically conduct white box testing on the different methods in the code. Covering the code and results after executing all the necessary testing. All the while using design patterns and SOLID principles. Finally, an evaluation of the implemented system compared to the original design seen in Report 1.

For this project, we also aim to maintain traceability between requirements, design, implementation, and tests.

1.1 Scope

The main emphasis of this project is placed on the core service layer functionality. This includes the prior mentioned project and activity management, resource allocation, progress tracking, and reporting. This is the most relevant part for detailed analysis, since it contains the central application logic. We also worked on creating a clean and professional graphical user interface (GUI) to realistically expedite the planned functionality of the system.

1.2 Relation to Report 1

This report builds on the analysis and design presented in Report 1. However, where Report 1 focused on requirements, use cases, and system design, this report focuses on the implemented solution and its evaluation.

During the development some implementation details were adjusted to keep the scope manageable, and to fulfill the requirement. The final system still reflects the central ideas and requirements identified in the earlier report.

2 Implemented System Overview (Vedanta Sinha)

This section gives an overview of the implemented system and structure. It focuses on the structure and functionality of the system. This section further aims to explain what the system does, how it does it, and how the structure of our code helps fulfill the required functionality.

2.1 Implemented Functionality

All the functionality of this system aims to help a company assign and finish projects in an orderly manner. The main functionalities include the creation of a project, creation and planning of activities, assignment of developers to activities, time registration on activities, absence registration, assignment of project leaders, project reporting, and progress tracking.

2.2 Architecture Overview

The system's architecture is made up from multiple layers, which are separated from each other logically based on their responsibilities. This makes them easier to test and scale.

These layers consist of the domain layer, which holds the core entities of the software, such as Projects, Activities, Developers, Absences, and Time Registration. Above the domain layer, one can find the repository layer, which functions as the repository. Its functionality is to manage data storage and retrieval. This specific iteration of the repository layer uses 'in-memory', which has its upside: easier coding, faster creation, retrieval, and storage of objects, since the memory operates in the RAM. It also has its downsides, since the repository will be cleared when the system boots off.

Building on top of the repository level's ability to store and retrieve objects, there is the service level. This is the layer tasked with applying logic and coordinating operations together so that the system can finish the tasks required. It is also tasked with ensuring that inputs are valid and consistent, and a real world company would use this layer to ensure that its employees couldn't break company policies.

On top of this is the User Interface, which is the interface that the user of the software uses in order to facilitate their commands. For this, there is a Graphical user interface being used, so that it is easiest for normal people in a company to use the software.

2.3 Traceability to Report 1

The system that is currently implemented is directly based on the the analysis done during Report 1, where sections such as the design and requirements of the software were refined. Overall, the implementation maintains consistency with the specifications and system design established in Report 1.

3 Systematic Tests (White-Box Tests)

In this section, we present systematic white-box tests for selected methods in the implemented system. Each group member selected one action-oriented method, as required by the project brief. Getter and setter methods were not selected, because they do not contain enough behaviour, branching, or validation logic to justify systematic white-box testing.

The test design follows the two table template from the course lectures. The first table identifies the relevant input properties for a method, such as valid and invalid values, boundary values, missing objects, and state combinations. The second table turns those properties into concrete test cases with specific input values and expected results. Each row in the concrete test table corresponds to an executable JUnit or Cucumber test.

The white-box tests complement the BDD tests. The Cucumber scenarios verify that the system supports the required user-level behavior, while the white-box tests focus on implementation-level paths inside specific methods. This is useful because the service layer contains the most important business rules, including validation, repository lookups, filtering, and state changes.

3.1 Choice of Methods

The following methods were selected:

- `ProjectService.createProject(...)`
- `ActivityService.createActivity(...)`
- `ActivityService.addDeveloperToActivity(...)`
- `TimeRegistrationService.registerTime(...)`
- `AbsenceService.registerAbsence(...)`

These methods were chosen because they represent key system behaviour and contain validation logic, state changes, or coordination across multiple classes.

3.2 `ProjectService.createProject(...)` (Vedanta)

3.2.1 Method

This is the most crucial method, as without a project, developers cannot be assigned, they cannot make reports, and activities cannot be given.

```
1 public Project createProject(String name) { // creates and then
    stores the created project
2     if (name == null || name.isBlank()) {
3         throw new IllegalArgumentException("Project name cannot
        be empty");
4     }
5     String id = projectRepository.generateProjectId();
6     Project project = new Project(id, name);
```

3.3 ActivityService.setActivityDetails(...) (Viktor)

```
7     projectRepository.add(project);
8     return project;
9 }
```

This small piece of code creates and stores valid

3.2.2 Input Properties

Property	Description
Project name valid	Non-empty project name
Project name invalid	Empty or blank project name

Tabel 1: Input properties for createProject(...)

3.2.3 Concrete Test Cases

Test case	Input	Expected result
TC1	"WebShop"	Project is created successfully
TC2	" "	Exception: project name cannot be empty

Tabel 2: Concrete test cases for createProject(...)

3.2.4 Implementation of Tests

The planned test cases were implemented as executable tests in the project. These tests verify both valid and invalid input behaviour.

3.3 ActivityService.setActivityDetails(...) (Viktor)

3.3.1 Method

```
1 public void setActivityDetails(String projectId, String activityName,
2                               double budgetedHours,
3                               int startWeek, int startYear,
4                               int endWeek, int endYear) {
5     if (budgetedHours < 0) {
6         throw new IllegalArgumentException(
7             "Budgeted hours cannot be negative");
8     }
9     if (startWeek != 0 && (startWeek < 1 || startWeek > 53)) {
10        throw new IllegalArgumentException(
11            "Start week must be between 1 and 53");
12    }
```


3.3 ActivityService.setActivityDetails(...) (Viktor)

```
12     }
13     if (endWeek != 0 && (endWeek < 1 || endWeek > 53)) {
14         throw new IllegalArgumentException(
15             "End week must be between 1 and 53");
16     }
17     if (startWeek != 0 && endWeek != 0) {
18         int start = startYear * 100 + startWeek;
19         int end = endYear * 100 + endWeek;
20         if (start > end) {
21             throw new IllegalArgumentException(
22                 "Start week must be before or equal to end week");
23         }
24     }
25     Activity activity = getActivity(projectId, activityName);
26     activity.setBudgetedHours(budgetedHours);
27     activity.setStartWeek(startWeek);
28     activity.setStartYear(startYear);
29     activity.setEndWeek(endWeek);
30     activity.setEndYear(endYear);
31 }
```

This method was chosen because it contains multiple independent validation branches (budget, start week range, end week range, and period ordering), making it well-suited for equivalence-class partitioning and boundary-value analysis. Getter and setter methods were deliberately excluded per the project brief.

3.3.2 Input Properties

Property	Description
Budget valid	$\text{budgetedHours} \geq 0$
Budget invalid	$\text{budgetedHours} < 0$
Start week unset	$\text{startWeek} = 0$ (no constraint applied)
Start week valid	$1 \leq \text{startWeek} \leq 53$
Start week invalid	$\text{startWeek} < 1$ or > 53
End week unset	$\text{endWeek} = 0$
End week valid	$1 \leq \text{endWeek} \leq 53$
End week invalid	$\text{endWeek} < 1$ or > 53
Period valid	$\text{start} \leq \text{end}$ (encoded as $\text{year} \times 100 + \text{week}$)
Period invalid	$\text{start} > \text{end}$

Tabel 3: Input properties for `setActivityDetails(...)`

3.4 ActivityService.addDeveloperToActivity(...)(Viktor)

3.3.3 Concrete Test Cases

TC	budgetedHours	startWeek / endWeek	Expected result
TC1	100.0	10 / 12 (2026)	Details set successfully
TC2	-1.0	10 / 12 (2026)	Exception: hours cannot be negative
TC3	0.0	0 / 0	Details set (unset weeks accepted)
TC4	50.0	1 / 53 (2026)	Details set (boundary weeks)
TC5	50.0	54 / 55 (2026)	Exception: start week out of range
TC6	50.0	10 / 54 (2026)	Exception: end week out of range
TC7	50.0	12 / 10 (2026)	Exception: start must be before end
TC8	50.0	5 (2026) / 3 (2025)	Exception: start must be before end

Tabel 4: Concrete test cases for setActivityDetails(...)

TC4 targets the boundary values 1 and 53 for both week parameters. TC8 verifies that the year is included in the ordering comparison: week 5 of 2026 as start and week 3 of 2025 as end raises an exception because $2026 \times 100 + 5 > 2025 \times 100 + 3$.

3.3.4 Implementation of Tests

The test cases above are implemented as JUnit 5 tests in `ActivityServiceTest.java`. Each case either asserts a successful state change on the activity object or verifies that the correct `IllegalArgumentException` message is thrown.

3.4 ActivityService.addDeveloperToActivity(...)(Viktor)

3.4.1 Method

```
1 public void addDeveloperToActivity(String projectId,
2                                   String activityName,
3                                   String developerInitials) {
4     Activity activity = getActivity(projectId, activityName);
5
6     Optional<Developer> result =
7         developerRepository.findByInitials(developerInitials);
8     if (!result.isPresent()) {
9         throw new IllegalArgumentException(
```

3.4 ActivityService.addDeveloperToActivity(...)(Viktor)

```
10         "Developer not found: " + developerInitials);
11     }
12     Developer developer = result.get();
13
14     activity.addDeveloper(developer);
15 }
```

3.4.2 Input Properties

Property	Description
Project exists	The project ID refers to an existing project in the repository.
Project does not exist	The project ID does not match any project in the repository.
Activity exists	The activity name refers to an existing activity in the selected project.
Activity does not exist	The activity name does not match an activity in the selected project.
Developer exists	The developer initials match a developer in the developer repository.
Developer does not exist	The developer initials are unknown to the system.
Developer already assigned	The developer is already assigned to the activity. The domain method prevents duplicate entries.

Tabel 5: Input properties for addDeveloperToActivity(...)

These properties cover both the successful path and the relevant error paths in the method. If the project or activity cannot be found, the method fails through getActivity(...). If the developer initials are unknown, the method throws an IllegalArgumentException. If all lookups succeed, the method performs the assignment.

3.5 TimeRegistrationService.registerTime(...)(Nat)

3.4.3 Concrete Test Cases

Test case	Input and system state	Expected result
TC1	Existing project, activity Requirements exists, developer huba exists.	Developer huba is assigned to activity Requirements.
TC2	Existing project, activity Requirements exists, developer initials zzzz do not exist.	IllegalArgumentException: Developer not found: zzzz.
TC3	Existing project, activity name Design does not exist, developer huba exists.	IllegalArgumentException: Activity not found: Design.
TC4	Project ID 99999 does not exist, activity Requirements is requested, developer huba exists.	IllegalArgumentException because the activity cannot be found for the project.
TC5	Existing project, activity Requirements exists, huba is added twice.	huba appears only once in the activity's assigned developer list.

Tabel 6: Concrete test cases for addDeveloperToActivity(...)

Together, the test cases exercise the method's main decision points: activity lookup, developer lookup, and the final state change on the Activity object. This makes the tests systematic rather than only example-based.

3.4.4 Implementation of Tests

The test cases are represented by the existing Cucumber scenarios for assigning developers to activities. The feature file contains a successful case where developer huba is added to the activity Requirements, and an error case where the unknown initials zzzz are rejected. These scenarios test the two most important branches in the method: successful lookup and assignment, and failed developer lookup.

The method is suitable for white-box testing because the result depends on internal repository lookups and on the state change performed on the Activity object. If the project or activity cannot be found, getActivity(...) throws an exception. If the developer cannot be found, addDeveloperToActivity(...) throws an exception. If both lookups succeed, the developer is added to the activity.

3.5 TimeRegistrationService.registerTime(...)(Nat)

3.5.1 Method

```
1 public TimeRegistration registerTime(String developerInitials, String  
   projectId,
```

3.5 TimeRegistrationService.registerTime(...)(Nat)

```
2                                     String activityName,
                                     LocalDate date, double
                                     hours) {
3
4     if (hours <= 0 || hours % 0.5 != 0) {
5         throw new IllegalArgumentException("Hours must be a
6             positive multiple of 0.5");
7     }
8
9     // Pre-conditions (hold after defensive validation above)
10    assert developerInitials != null : "developerInitials must
11        not be null";
12    assert projectId != null : "projectId must not be null";
13    assert activityName != null : "activityName must not be null"
14        ;
15    assert date != null : "date must not be null";
16    assert hours > 0 : "hours must be positive";
17    assert hours % 0.5 == 0 : "hours must be a multiple of 0.5";
18
19    Developer developer = developerRepository.findByInitials(
20        developerInitials)
21        .orElseThrow(() -> new IllegalArgumentException("
22            Developer not found: " + developerInitials));
23
24    Activity activity = projectRepository.findActivity(projectId,
25        activityName)
26        .orElseThrow(() -> new IllegalArgumentException("
27            Activity not found: " + activityName));
28
29    TimeRegistration registration = new TimeRegistration(
30        developer, activity, date, hours);
31    activity.addTimeRegistration(registration);
32
33    // Post-conditions
34    assert activity.getTimeRegistrations().contains(registration)
35        : "registration must be stored on activity";
36    assert registration.getHours() == hours : "registered hours
37        must match input";
38
39    return registration;
40 }
```

[]

3.6 `AbsenceService.registerAbsence(...)`

3.5.2 Input Properties

Property	Description
Hours valid	Positive number and multiple of 0.5
Hours zero or negative	Invalid number of hours
Hours not multiple of 0.5	Invalid granularity
Activity exists	The activity exists in the specified project

Tabel 7: Input properties for `registerTime(...)`

3.5.3 Concrete Test Cases

Test case	Input	Expected result
TC1	2.5 hours	Time registration stored successfully
TC2	0.0 hours	Exception: invalid number of hours
TC3	1.3 hours	Exception: must be multiple of 0.5

Tabel 8: Concrete test cases for `registerTime(...)`

3.6 `AbsenceService.registerAbsence(...)`

3.6.1 Method

[Insert method code here]

3.6.2 Input Properties

Property	Description
Developer is free in period	Absence can be registered
Developer is busy in period	Absence must be rejected
Valid week interval	Start and end period form a meaningful interval

Tabel 9: Input properties for `registerAbsence(...)`

3.6.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Vacation in free period	Absence registered successfully
TC2	Vacation in already booked period	Exception: vacation denied

Tabel 10: Concrete test cases for `registerAbsence(...)`

3.7 Summary of White-Box Testing

The systematic tests were derived from input partitions and boundary-relevant cases. The goal was to ensure that each selected method was tested for both normal behaviour and relevant error conditions. These white-box tests complement the existing BDD tests and provide more explicit validation of implementation logic.

4 Design by Contract ()

This section presents contracts for the same methods used in the systematic testing section. For each method, some preconditions are assumed, and postconditions are set. This is important as a logical system like code requires a systematic setup.

4.1 ProjectService.createProject(...)(Vedanta Sinha)

4.1.1 Method with Assertions

```
1 public Project createProject(String name) { // creates and then
    stores the created project
2     if (name == null || name.isBlank()) {
3         throw new IllegalArgumentException("Project_name_cannot_be_
            empty");
4     }
5     String id = projectRepository.generateProjectId();
6     Project project = new Project(id, name);
7     projectRepository.add(project);
8     return project;
9 }
```

4.1.2 Contract

Precondition:

$$name \neq null \wedge name.trim().length() > 0$$

Postcondition:

$$result \neq null \wedge result.getName() = name$$

4.1.3 Discussion

The precondition ensures that only valid project names are accepted. The postcondition guarantees that a project object is created and stored consistently.

4.2 ActivityService.setActivityDetails(...)(Viktor)

4.2.1 Method with Assertions

```
1 public void setActivityDetails(String projectId, String activityName,
2                               double budgetedHours,
3                               int startWeek, int startYear,
4                               int endWeek, int endYear) {
5     if (budgetedHours < 0) {
```


4.2 ActivityService.setActivityDetails(...) (Viktor)

```
6         throw new IllegalArgumentException(  
7             "Budgeted_hours_cannot_be_negative");  
8     }  
9     if (startWeek != 0 && (startWeek < 1 || startWeek > 53)) {  
10         throw new IllegalArgumentException(  
11             "Start_week_must_be_between_1_and_53");  
12     }  
13     if (endWeek != 0 && (endWeek < 1 || endWeek > 53)) {  
14         throw new IllegalArgumentException(  
15             "End_week_must_be_between_1_and_53");  
16     }  
17     // check that start is not after end  
18     if (startWeek != 0 && endWeek != 0) {  
19         int start = startYear * 100 + startWeek;  
20         int end = endYear * 100 + endWeek;  
21         if (start > end) {  
22             throw new IllegalArgumentException(  
23                 "Start_week_must_be_before_or_equal_to_end_week");  
24         }  
25     }  
26  
27     assert budgetedHours >= 0  
28         : "budgetedHours_must_be_non-negative";  
29     assert startWeek == 0 || (startWeek >= 1 && startWeek <= 53)  
30         : "startWeek_must_be_0_or_between_1_and_53";  
31     assert endWeek == 0 || (endWeek >= 1 && endWeek <= 53)  
32         : "endWeek_must_be_0_or_between_1_and_53";  
33  
34     Activity activity = getActivity(projectId, activityName);  
35     activity.setBudgetedHours(budgetedHours);  
36     activity.setStartWeek(startWeek);  
37     activity.setStartYear(startYear);  
38     activity.setEndWeek(endWeek);  
39     activity.setEndYear(endYear);  
40  
41     assert activity.getBudgetedHours() == budgetedHours  
42         : "budgeted_hours_must_match_input";  
43     assert activity.getStartWeek() == startWeek  
44         : "start_week_must_match_input";  
45     assert activity.getEndWeek() == endWeek  
46         : "end_week_must_match_input";  
47 }
```

4.3 ActivityService.addDeveloperToActivity(...)(Viktor)

4.2.2 Contract

Precondition:

$$\text{budgetedHours} \geq 0 \wedge (\text{startWeek} = 0 \vee 1 \leq \text{startWeek} \leq 53) \wedge (\text{endWeek} = 0 \vee 1 \leq \text{endWeek} \leq 53)$$
$$\wedge (\text{startWeek} \neq 0 \wedge \text{endWeek} \neq 0) \Rightarrow \text{startYear} \times 100 + \text{startWeek} \leq \text{endYear} \times 100 + \text{endWeek}$$

Postcondition:

$$\begin{aligned} &\text{activity.getBudgetedHours()} = \text{budgetedHours} \\ &\wedge \text{activity.getStartWeek()} = \text{startWeek} \\ &\wedge \text{activity.getEndWeek()} = \text{endWeek} \end{aligned}$$

4.2.3 Discussion

The precondition captures the four independent constraints that must hold before the method may safely mutate the activity, non negative budget, both week parameters either unset (0) or within the ISO week number range 1–53, and a chronological ordering of the period when both endpoints are provided. The value 0 serves as a sentinel meaning “not yet planned”, which is an explicit requirement from the project description.

The postcondition guarantees that all three planning fields on the activity object reflect the supplied arguments after a successful invocation. The assert-statements in the code enforce both conditions at runtime when the JVM is started with `-ea`.

4.3 ActivityService.addDeveloperToActivity(...)(Viktor)

4.3.1 Method with Assertions

```
1 public void addDeveloperToActivity(String projectId,
2                                   String activityName,
3                                   String developerInitials) {
4     assert projectId != null : "projectId_must_not_be_null";
5     assert activityName != null : "activityName_must_not_be_null";
6     assert developerInitials != null
7         : "developerInitials_must_not_be_null";
8
9     Activity activity = getActivity(projectId, activityName);
10    int assignedBefore = activity.getAssignedDevelopers().size();
11
12    Optional<Developer> result =
13        developerRepository.findByInitials(developerInitials);
14    if (!result.isPresent()) {
15        throw new IllegalArgumentException(
16            "Developer_not_found: " + developerInitials);
17    }
18    Developer developer = result.get();
```

4.3 ActivityService.addDeveloperToActivity(...)(Viktor)

```
19
20     activity.addDeveloper(developer);
21
22     assert activity.getAssignedDevelopers().contains(developer)
23           : "developer must be assigned to the activity";
24     assert activity.getAssignedDevelopers().size() >= assignedBefore
25           : "assignment must not remove developers";
26 }
```

4.3.2 Contract

Precondition:

$$projectId \neq null \wedge activityName \neq null \wedge developerInitials \neq null$$

$$projectExists(projectId) \wedge activityExists(projectId, activityName) \wedge developerExists(developerInitials)$$

Postcondition:

$$developer \in activity.getAssignedDevelopers()$$

$$activity.getAssignedDevelopers()_{after}.size() \geq activity.getAssignedDevelopers()_{before}.size()$$

$$activity = projectRepository.findActivity(projectId, activityName)$$

$$developer = developerRepository.findByInitials(developerInitials)$$

4.3.3 Discussion

The precondition captures the constraints that must hold before the method may safely assign a developer to an activity. The three string parameters must not be null, the project must exist, the activity must exist inside that project, and the developer initials must refer to a known developer in the system. These conditions match the purpose of the method: it should only connect already existing domain objects, not create new projects, activities, or developers.

The postcondition guarantees the state change caused by a successful invocation. After the method has run without throwing an exception, the selected developer is contained in the activity's assigned developer list. The size condition states that the operation must not remove existing developers from the activity. If the developer was not already assigned, the list grows by one. If the developer was already assigned, the Activity class prevents duplicates,

4.4 TimeRegistrationService.registerTime(Nat)

so the list may keep the same size while still satisfying the contract.

The assert statements in the code express the same idea at runtime when the JVM is started with `-ea`. The precondition assertions document the required inputs, while the postcondition assertions check that the assignment has actually been stored on the activity. This makes the contract useful both as documentation and as a runtime check during testing.

4.4 TimeRegistrationService.registerTime(Nat)

4.4.1 Method with Assertions

Listing 1: Register Time Method

```
1 public TimeRegistration registerTime(String developerInitials, String
   projectId,
2                                     String activityName,
                                       LocalDate date, double
                                       hours) {
3     if (hours <= 0 || hours % 0.5 != 0) {
4         throw new IllegalArgumentException("Hours must be a
           positive multiple of 0.5");
5     }
6
7     // Pre-conditions (hold after defensive validation above)
8     assert developerInitials != null : "developerInitials must
           not be null";
9     assert projectId != null : "projectId must not be null";
10    assert activityName != null : "activityName must not be null"
        ;
11    assert date != null : "date must not be null";
12    assert hours > 0 : "hours must be positive";
13    assert hours % 0.5 == 0 : "hours must be a multiple of 0.5";
14
15    Developer developer = developerRepository.findByInitials(
        developerInitials)
16        .orElseThrow(() -> new IllegalArgumentException("
           Developer not found: " + developerInitials));
17
18    Activity activity = projectRepository.findActivity(projectId,
        activityName)
19        .orElseThrow(() -> new IllegalArgumentException("
           Activity not found: " + activityName));
20
21    TimeRegistration registration = new TimeRegistration(
        developer, activity, date, hours);
22    activity.addTimeRegistration(registration);
```

4.4 TimeRegistrationService.registerTime(Nat)

```
23
24 // Post-conditions
25 assert activity.getTimeRegistrations().contains(registration)
26 : "registration_must_be_stored_on_activity";
27 assert registration.getHours() == hours : "registered_hours_
28 must_match_input";
29
30 return registration;
31 }
32 #####
33 public TimeRegistration registerTime(String developerInitials,
34                                     String projectId,
35                                     String activityName,
36                                     LocalDate date,
37                                     double hours) {
38
39     if (hours <= 0 || hours % 0.5 != 0) {
40         throw new IllegalArgumentException("Hours_must_be_a_positive_
41         multiple_of_0.5");
42     }
43
44     Developer developer = developerRepository.findByInitials(
45         developerInitials)
46         .orElseThrow(() -> new IllegalArgumentException(
47             "Developer_not_found:_" + developerInitials));
48
49     Activity activity = projectRepository.findById(projectId)
50         .orElseThrow(() -> new IllegalArgumentException(
51             "Project_not_found:_" + projectId))
52         .getActivities().stream()
53         .filter(a -> a.getName().equals(activityName))
54         .findFirst()
55         .orElseThrow(() -> new IllegalArgumentException(
56             "Activity_not_found:_" + activityName));
57
58     TimeRegistration registration =
59         new TimeRegistration(developer, activity, date, hours);
60
61     activity.addTimeRegistration(registration);
62
63     return registration;
64 }
```

4.4.2 Contract

Precondition:

$$hours > 0 \wedge hours \bmod 0.5 = 0$$

$$projectExists(projectId) \wedge activityExists(projectId, activityName) \wedge developerExists(initials) \\ developer \in activity.assignedDevelopers$$

Postcondition:

$$\exists r \in activity.timeRegistrations : r.hours = hours$$

4.5 Discussion

The precondition ensures that only valid amounts of time can be added to existing activities that the developer is added to.// The postcondition ensures that the time has been registered correctly.

4.6 AbsenceService.registerAbsence(...)

4.6.1 Method with Assertions

Listing 2: Register Absence Method

```

1  public Absence registerAbsence(String developerInitials,
2                                Absence.Type type,
3                                int startWeek, int startYear,
4                                int endWeek, int endYear) {
5
6
7      Developer developer = developerRepository.findByInitials(
8          developerInitials)
9          .orElseThrow(() -> new IllegalArgumentException(
10              "Developer not found: " + developerInitials));
11
12      if (type == Absence.Type.VACATION &&
13          isDeveloperBusyInPeriod(developer, startWeek, startYear,
14                                  endWeek, endYear)) {
15
16          throw new IllegalArgumentException(
17              "Vacation denied: developer is assigned to an
18              activity in this period");
19      }
20
21      Absence absence = new Absence(developer, type,

```

```

19         startWeek, startYear,
20         endWeek, endYear);
21
22     absenceRepository.add(absence);
23
24     return absence;
25 }

```

4.6.2 Contract

Precondition:

$developerExists(initials) \wedge periodIsValid(startWeek, startYear, endWeek, endYear)$

Postcondition:

$absenceRegistered \vee exceptionThrown$

4.7 Discussion of Contracts

The contracts make assumptions and guarantees explicit. They improve readability, support debugging, and document the intended behaviour of the methods. In our implementation, assertions were added to enforce preconditions and postconditions directly in code.

5 Code Coverage (Viktor)

Code coverage is a quantitative metric that expresses the proportion of production code exercised at least once during a test suite execution. As Edsger W. Dijkstra famously noted, “testing can only reveal the presence of bugs, never their absence” coverage does not resolve this fundamental limitation, but it transforms an otherwise opaque question into a concrete, measurable property. A low coverage figure therefore guarantees that some code is untested, while a high figure is a necessary but not sufficient condition for confidence in the suite.

In the context of this project, the analysis serves two concrete purposes: to validate that the TDD workflow was genuinely followed in the service layer, and to document the concrete effect of adding the systematic white-box tests from Section 3.

5.1 Measurement Setup

Coverage was collected using Visual Studio Code’s built in "Run Tests with Coverage" action in the Test Explorer. VS Code’s Java Test Runner extension bundles the JaCoCo agent and attaches it to the JVM at test time, so no JaCoCo entry is required in `pom.xml`. JaCoCo performs on-the-fly bytecode instrumentation, probes are inserted when each class is loaded, recording which instructions and branches are executed. The results appear in VS Code’s `test coverage` panel.

The suite that underpins the measurement consists of 57 passing tests: 32 Cucumber BDD scenarios covering 12 use cases, and 25 JUnit white-box tests, 14 in `ProjectServiceTest`, 11 in `ActivityServiceTest` designed using equivalence partitioning and boundary value analysis.

5.2 Results

Table 11 summarises the per-package figures. The overall weighted total of 29% is dominated by the UI layer, which accounts for 880 of the 1245 executable lines and has 0% coverage by design (discussed in Section 5.3).

Package	Line coverage	Lines
<code>com.planner.domain</code>	79.77%	105
<code>com.planner.repository</code>	89.16%	53
<code>com.planner.service</code>	77.25%	207
<code>com.planner.ui</code>	0.00%	880
Total (weighted)	29.00%	1245

Tabel 11: Line coverage per package as reported by JaCoCo via VS Code’s `test coverage` panel.

Figure 1 shows the top-level `test coverage` panel after running all 57 tests. The panel

is structured hierarchically from the `com.planner` root down through sub packages, only the package level is shown here, with the per class views presented in Section 5.3.

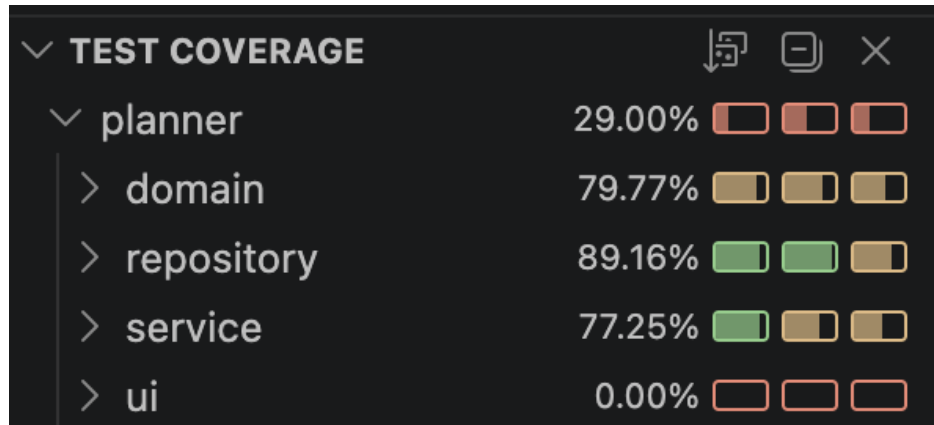


Figure 1: VS Code test coverage panel at package level after executing all 57 tests.

5.3 Analysis by Layer

Repository Layer

Figure 2 shows the per-class breakdown of the repository layer. `ProjectRepository.java` achieves 100% line coverage, as it is reached on every Cucumber scenario that creates or looks up a project. `AbsenceRepository.java` reaches 94.44%, with the small gap stemming from a defensive branch that handles a missing or empty absence list. `DeveloperRepository.java` sits at 79.49%; the uncovered lines are in the `loadFromFile` fallback that inserts `huba` when the resource file cannot be read — a branch that is unreachable under normal test conditions and is therefore intentionally left unexercised.

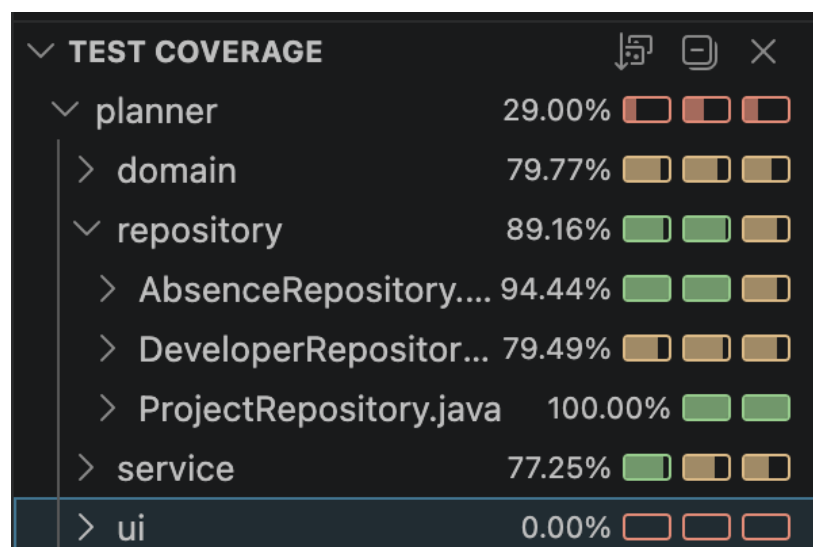


Figure 2: Per-class coverage within `com.planner.repository`.

UI Layer

Figure 3 shows the per-class breakdown of the UI layer. All four classes — `App.java`, `AppContext.java`, `ConsoleUI.java`, and `MainWindow.java` — report 0.00% coverage. This is a deliberate consequence of the project’s testing strategy: the project brief explicitly states that the user interface does not need to be developed test-first, and Robert C. Martin’s argument that the presentation layer sits at the edge of the system applies directly here. Every meaningful action in the UI delegates immediately to a service-layer method that is already covered by the Cucumber and JUnit test suites. The UI layer is therefore tested manually through the demonstration on 11 May rather than through automated tests.

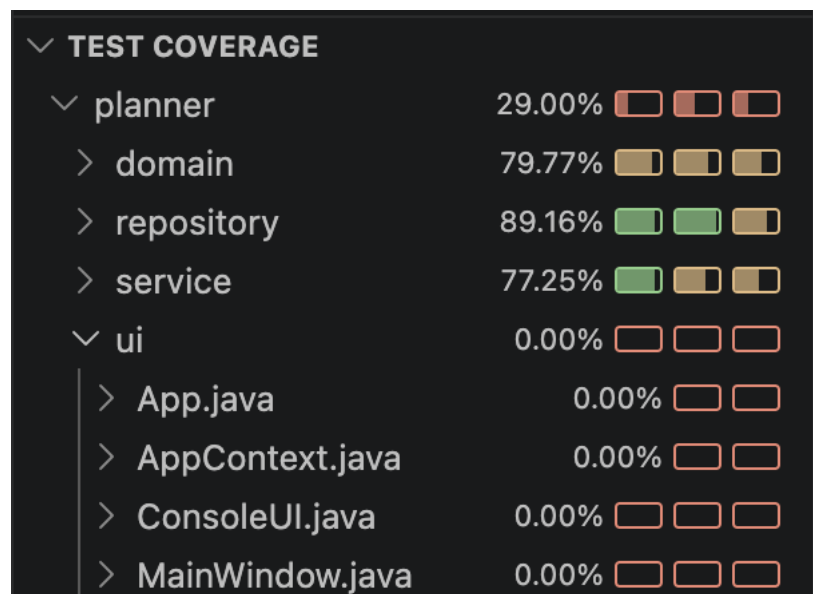
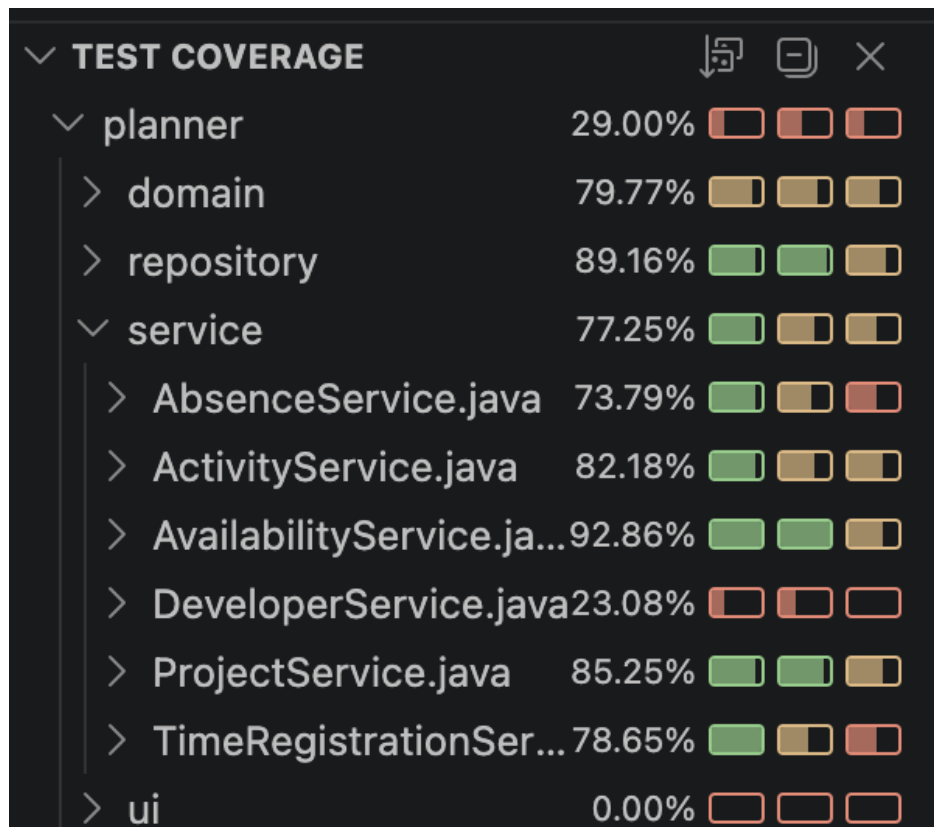


Figure 3: Per-class coverage within `com.planner.ui`.

5.3.1 Service layer 77.25%

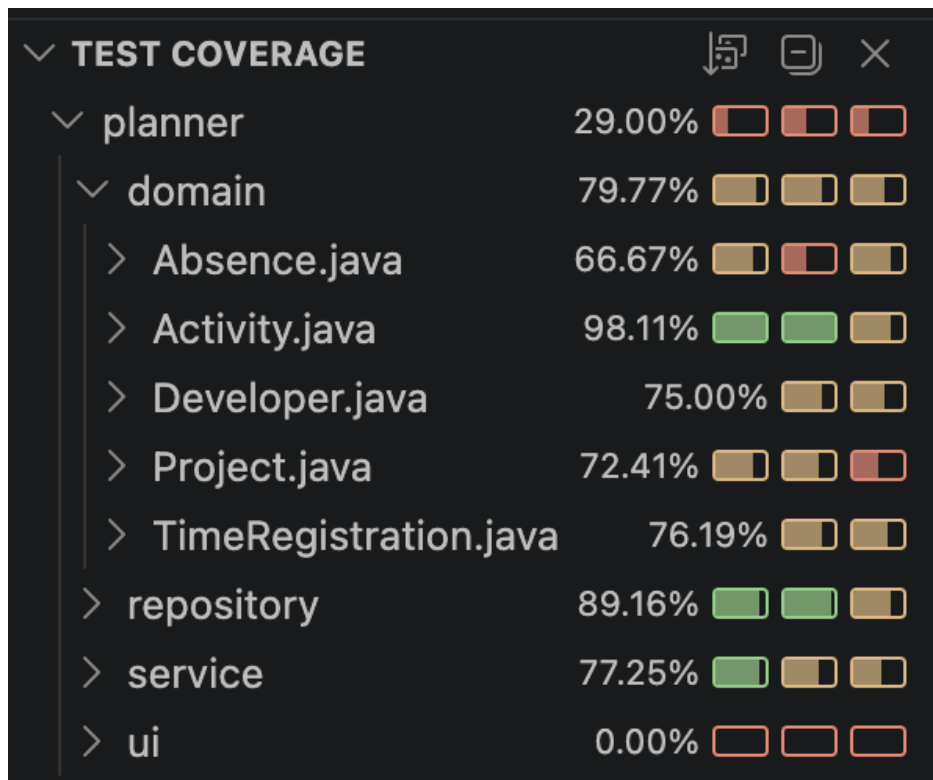
This layer was developed strictly test first, so every public method in the six services has at least one Cucumber scenario. Figure 4 shows the per class breakdown. The 25 JUnit tests raise branch coverage further on `ProjectService` (85.25%) and `ActivityService` (82.18%) by targeting boundary values and edge cases, null/empty/whitespace names, week numbers 0, 1, 53, 54 that the BDD scenarios do not reach. `DeveloperService` sits notably lower at 23.08% because the project brief specifies that developers are imported from the HR file rather than created through the system, leaving most of its methods unused in any scenario.



Figur 4: Per-class coverage within `com.planner.service`.

5.3.2 Domain layer 79.77%

Domain objects are thin data holders whose behaviour is exercised indirectly through the service tests. Figure 5 shows the per-class breakdown. `Activity.java` reaches 98.11% because activities are manipulated in almost every use case, while `Absence.java` stops at 66.67%, reflecting the smaller number of absence-related scenarios. The gap from 100% consists entirely of trivial getters, `toString`, `getType`, `setDate` that no test calls directly; the project brief explicitly advises against testing bare getters.



Figur 5: Per-class coverage within `com.planner.domain`.

5.4 Effect of the White-Box Tests

The Cucumber scenarios verify business behaviour from the outside: they describe happy paths and the most prominent error cases in natural language, and they exercise whole call chains from the service layer down into the domain. Because they are written in terms of business outcomes, they are not designed to target individual branches or boundary values inside a method.

The 25 JUnit tests introduced in Section 3 fill exactly this gap. Using equivalence partitioning and boundary value analysis, they target the code's internal control-flow structure: the boundary values 0, 1, 53 and 54 in week-number validation; the three separate null/empty/whitespace cases for project names; and the conflict check between a new absence and an existing activity assignment. These are branches the Cucumber suite rarely or never reaches.

The combined suite produces a visibly tighter branch coverage on `ProjectService` and `ActivityService` than either approach achieves alone. This is not a coincidence; it is the empirical confirmation of the standard software engineering claim that black box specification-based testing and white box structural testing are complementary methodologies, each reaching parts of the code the other leaves untouched.

5.5 Limitations

Coverage measures *execution*, not *correctness*. A test that calls a method without asserting anything contributes fully to coverage while verifying nothing. The figures in Table 11 should therefore be read alongside the Cucumber scenarios, behavioural correctness, the JUnit assertions (boundary correctness), and the design-by-contract checks in Section 4. Taken together, these four sources of evidence provide a considerably stronger argument for test adequacy than coverage alone.

5.6 Summary

Table 12 provides a compact overview of the main findings.

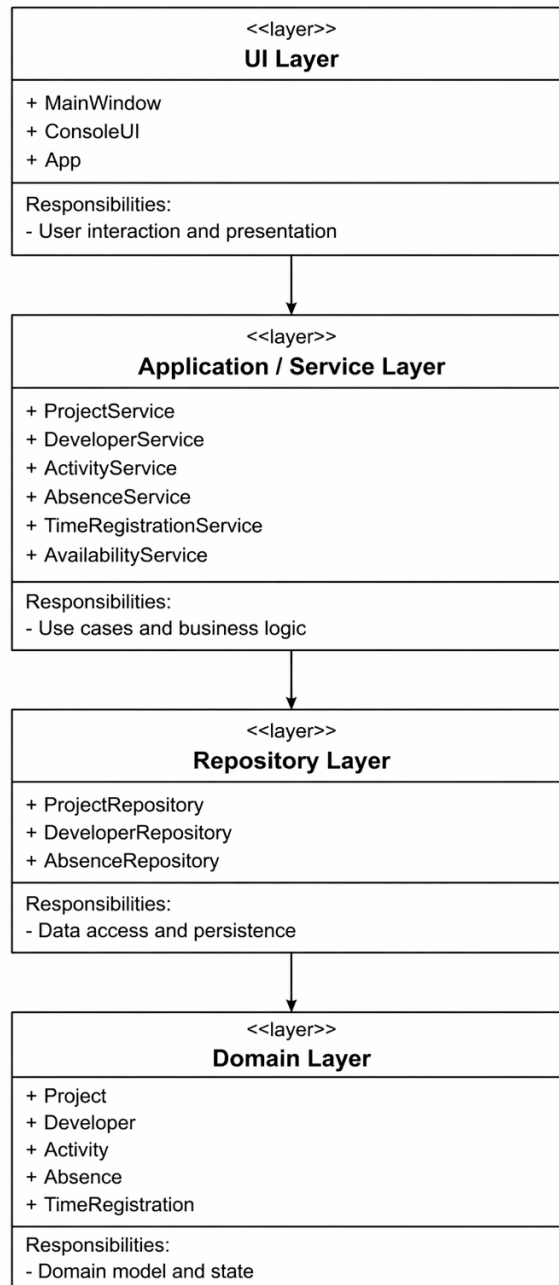
Layer	Coverage	Interpretation
Service	High (77.25%)	TDD followed; white-box tests close remaining branch gaps
Domain	Medium (79.77%)	Thin data classes; uncovered code is trivial getters only
Repository	High (89.16%)	Reached transitively; gap is interface bytecode artefact
UI	None (0.00%)	Deliberately excluded; business logic tested at service level

Tabel 12: Summary of coverage findings per architectural layer.

6 Design Patterns and SOLID Principles (Nicolai)

6.1 Overall Design Choices

This project is centered around a layered architecture, where the code is separated into clear and separate *domain-classes*, *services*, *repositories*, and *user interface* as illustrated in 6



Figur 6: The layered achitecture of the system

This was done to create clear separation between files to make it possible for the group to work in parallel in tandem without too many dependencies between parts. Clarity, separation and function-independence has been in the groups design-scope from the start.

6.1.1 Domain Layer

The domain-layer carries the central objects of the program and includes the domain-files *Project*, *Activity*, *Developer*, *Absence* and *TimeRegistration*. These classes represent all data and relations within the system, and the program as a whole depends solely on them for the overall state of the system. User interaction, storage and higher application logic such as system rules, are kept out of this layer, to minimize dependencies to other parts of the system and avoid coupling between layers. For example, the *Project* class contains basic project information and a relation to its activities:

```

1      public class Project {
2          private final String id;
3          private final String name;
4          private Developer projectLeader;
5          private final List<Activity> activities = new ArrayList<>();
6
7          public Project(String id, String name) {
8              this.id = id;
9              this.name = name;
10         }
11     }

```

This way, the domain classes only store information about the given projects, and the relation to other domain objects, keeping presentation, general data storage and system rules out of the domain layer. These functions and responsibilities are handled by the service, repository and interface layers.

6.1.2 Repository Layer

The repository-layer is used to isolate general data storage from the rest of the system. The classes, *ProjectRepository*, *DeveloperRepository* and *AbsenceRepository* are in charge of saving and gathering data from the other layers in the system. In final iterations of the project, in-memory storage was introduced into these classes, keeping objects inside the repositories for easier testing. For an example, in the project repository

```

1      public class ProjectRepository {
2          private final List<Project> projects = new ArrayList<>();
3
4          public void add(Project project) {
5              projects.add(project);
6          }
7      }

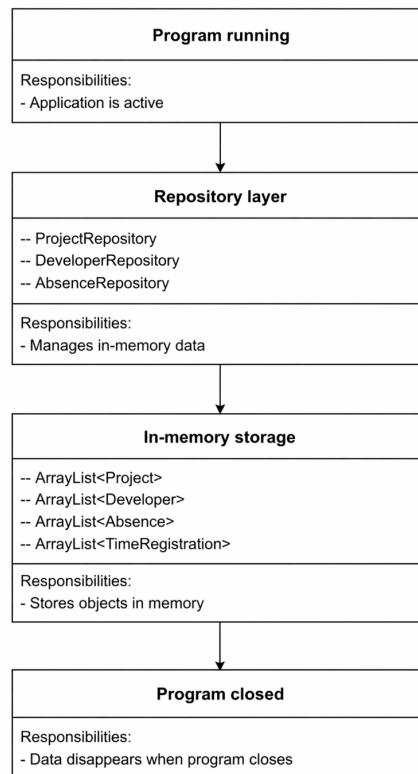
```

```

8      public Project findById(String id) {
9          // searches the list of projects
10     }
11 }

```

This way data is kept inside the single program as seen in 7



Figur 7: In-memory repositories

This was done as a deliberate design choice to support system testing as well as application logic for the group and further development, rather than having a persistent storage introducing unnecessary complexity for the required functionality of the program.

6.1.3 Service layer

While the state of the system is upheld and kept by the domain-layers, and data storage is kept by the repositories, the application logic is solely placed in the systems service layers. The layer, consisting of the classes *ProjectService*, *ActivityService*, *TimeRegistrationService*, and *AbsenceService*, each handle their own specific part of the systems functionality. This layer is responsible for coordinating all operations within the project, enforcing system rules, and keeping developers, leaders and projects aligned with eachother. For example, the creation of an activity is handled in the *ActivityService*:

Listing 3: Activity creation in service layer

```

1 public void createActivity(String projectId, String activityName) {
2     Project project = projectRepository.findById(projectId);
3     // validation and logic
4     project.addActivity(new Activity(activityName));
5 }

```

As seen in the listing above, the *ActivityService* is end the *projectId* and *ActivityName*, then finds the correct project in the repository, validates and applies the necessary logic needed. The domain object itself does not handle how activities are created, and the repository is only responsible for storing and retrieving data.

6.1.4 User Interface Layer

The user interface layer is the interaction between the program and end user. This interface consist of a console-based interface and a graphical JavaFX-based interface. The UI layer is where all before mentioned logic is used or presented to the user. User input, and program functionality happens at this level, while keeping the actual logic hidden. Instead input is delegated to the before mentioned services, *ProjectService*, *ActivityService*, and *AbsenceService*. For an example, if a user wants to create a new project through the interface, the UI collects the input and call the relevant service-layer method needed. Validation, System logic and updates are then kept in these layers, leaving the interface layer as prestation and input only.

```

1 projectService.createProject(projectName);

```

This way, all system logic is kept from the end user, all while keeping all necessary information viewable in the *MainWindow*. It should be noted this is the overall politics of the interface, as there is quite an amount of UI-related code viewable instead of it being a full out developed GUI architecture. This was due to it being one of the last features developed, and was agreed upon as acceptable trade-off for focus on the project service and domain logic.

6.1.5 Overall Design Considerations

It should be noted that the layered architecture was chosen because it was the best way to balance between structure, clarity and simplicity. While more advanced architectural patterns could have been applied, this approach was considered sufficient for the scope of the project.

The design prioritizes ease of development, making the system easier to test, maintain and extend without introducing unnecessary complexity.

6.2 SOLID-principles

In this project, focus has been on a structured system with clarity, testability, and further development in mind. As described in the layered architecture above, the system is already separated into *domain-classes*, *services*, *repositories*, and *user interface*, which naturally

aligns with several SOLID principles.

We have used the SOLID-principles to the extent we found useful for a project of this size and scope. Readability and understanding of the code has been prioritized, rather than forcing design patterns where they did not solve a specific problem.

The layered architecture, repository pattern, and dependency injection are among the design choices made, while principles such as SRP and DIP are the most prominent in the final system.

6.3 Single Responsible Principle

The program is highly structured around the Single Responsibility Principle through the layered design described earlier. Each part of the system has a clear primary responsibility, which contributes to a more clear and maintainable codebase.

As illustrated in Figure 6, the separation between the layers is prominent, where each layer is responsible for a distinct part of the system, and the functionality is distributed accordingly.

6.3.1 Domain-classes

The layered structure and separation is easy to spot in the projects classes. Looking into the domain-classes, *Project*, *Activity*, *Developer*, *Absence* and *TimeRegistration*, we see that they primarily represents the relations and data within the system. Together, they keep system truth aligned and ensures the consistency of the data. They maintain the state that the program actually works within, making the project work.

For example, a *Project* contains a list of activities and basic project information:

```

1  public class Project {
2      private String name;
3      private Developer projectLeader;
4      private List<Activity> activities;
5  }
```

On the other hand, these classes are deliberately kept independent from the end user and data storage responsibilities. Data storage is instead handled by the repository layer.

6.3.2 Repositories

The repository layer is divided into classes such as *ProjectRepository*, *DeveloperRepository* and *AbsenceRepository*, each managing their own specific part of system-data. For an instance, the *projectRepository* is responsible for storing and retrieving the different projects inside the system:

```

1  public class ProjectRepository {
2      private List<Project> projects = new ArrayList<>();
```

```

3
4     public void add(Project project) {
5         projects.add(project);
6     }
7
8     public Project findById(String id) {
9         // lookup logic
10    }
11 }

```

Separation between the two layers, *domain and repositories*, ensures that changes to data storage does not directly affect the logic of the domain logic. This does not only make the system more manageable, but also flexible for further development if needed.

6.3.3 Service layer

The service layer coordinates operations between the domain and repository layers, while enforcing system rules and logic.

Through the layers just described, *Domain, Repository and Service*, it is clearly shown how responsibilities are separated. This directly reflects the Single Responsibility Principle, where each layer has a clear and limited role in the overall system.

6.4 Dependency Inversion Principle

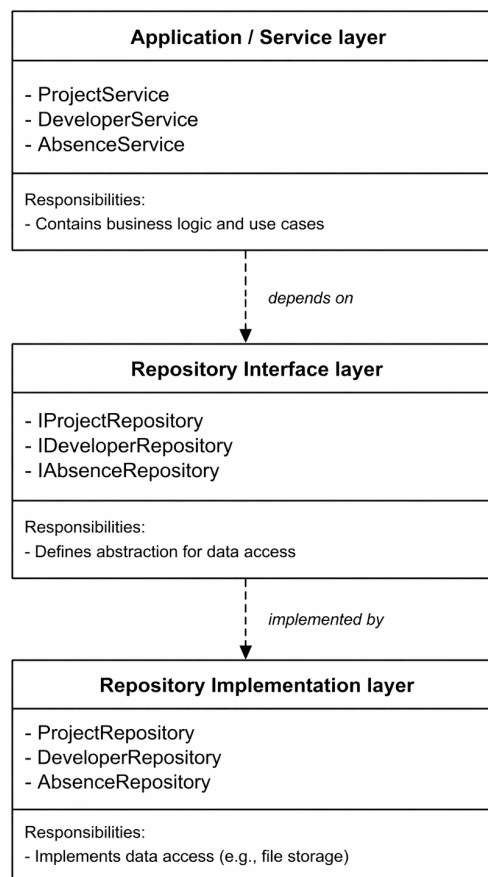
During late development of the final version of the project, the service layer was improved even further by applying repository interfaces such as *IProjectRepository*, *IAbsenceRepository* and *IDeveloperRepository*, using another SOLID principle, Dependency Inversion Principle. This was done to improve the overall design and future maintainability, with these introduced interfaces, service classes, *ProjectService*, *AbsenceService* and *DeveloperService*, no longer depend on the repository implementation, but use the *I-repositories* as abstractions. An example of this, can be seen in the constructor in the three serviceclasses, here *ProjectService*

```

1     public class ProjectService {
2         private final IProjectRepository projectRepository;
3
4         public ProjectService(IProjectRepository projectRepository) {
5             this.projectRepository = projectRepository;
6         }
7     }

```

This design choice, with DIP in mind, reduced the general dependency the service and storage had on each other. With this alteration, the general repository implementation happens outside the class, instead of being created internally. This makes the design more flexible in further development, because it makes it possible to replace general repository implementations without the need for change in the service logic itself.



Figur 8: DIP-flow (skal renames er træet)

6.5 Other SOLID-Principles

Single Responsibility and Dependency Inversion, are by design the most prominent and envisioned principles used in the Project. The remaining principles in SOLID, *Open/closed*, *Interface Segregation* and *Liskov* are only partially used in the design.

6.5.1 OCP, Open/Close principle

The Open/close principle, which is about the code being open for extension, but closed to modification, is only partially used in the service layer. Here, new functionality can be coded through service methods without actually modifying the domain classes directly.

```

1      public class ProjectService {
2
3      public void createProject(String name) {
4          // existing functionality
5      }
6
7      // New functionality
8      public void calculateProgress(String projectId) {
9          Project project = projectRepository.findById(projectId);
10         -> //new logic added without modifying Project class
11     }
12 }
```

This only points in the direction of real OCP, because of the need to change the service class to add functionalities to the code, and it can be concluded that the system is not designed around OCP extensibility.

The scope the group have had throughout the coding process has been on clarity, testability, and well structured understandable code that supports the structured implementation of the core functionality. To fully implement and support the open/close principle, would require a design overhaul, and result in complexity which we did not consider necessary for the overall purpose and functionality of the system we already build.

6.5.2 Interface Segregation Principle

The Interface Segregation states, that clients should not depend on functionality they do not need. Instead, the program should separate out the needed functionality in interfaces. In our program, ISP is not that prevalent. We did not feel the need to split the functionality up into separate interfaces more than it already is. Every client, the developer or project leader, have a very little interface already to begin with. The only place where ISP is somewhat used is again in the before mentioned repository interfaces, Separate out needed functionality in interfaces, *IProjectRepository*, *IAbsenceRepository* and *IDeveloperRepository*. However, this structure primarily follows the overall repository design, rather than being a direct result of applying ISP as a design principle.

6.5.3 Liskov Substitution Principle

At last the Liskov Substitution Principle, which states that subclasses should be changeable for their general base classes without affecting the functionality and correctness of the overall program. Or in simpler terms, classes should be able to inherit from other subclasses without change in the code. In our project, LSP is not used in the design. The system we made do not rely on inheritance of class hierarchies, but instead primarily focuses on composition and clear separation between the layers. As a result there are very few functions where substitution between subclasses are relevant, and the Liskov Substitution Principle are therefore not applied in the system intentionally. This reflects the overall design choice the group made to begin with, where composition and clarity was in a higher priority.

6.5.4 Final remarks

Overall, the use of SOLID principles in this project has been pragmatic rather than strict. The focus has been on creating a clear and maintainable structure, where the most relevant principles, such as SRP and DIP, have been applied where they added value. Other principles have only been partially used or not applied, as the added complexity was not considered necessary for the scope and functionality of the system.

7 Conclusion

The final system implements the core functionality required for project planning, activity management, developer assignment, time registration, absence handling, and project tracking.

The implemented product is consistent with the main requirements selected in Report 1, although some parts were simplified to keep the scope manageable. In particular, the final implementation focuses on a coherent in-memory solution with automated tests and a working user interface.

Compared to the design in Report 1, the final implementation follows the same main structure, but some adjustments were made during implementation. These changes improved feasibility and reduced complexity while preserving the key system behaviour.

The testing work consists of both BDD-style feature tests and additional systematic white-box tests. Together, these provide stronger confidence in the correctness of the core application logic.

7.1 Reflection on the Development Process

The project was developed collaboratively across the group. Work was divided across analysis, implementation, testing, and documentation tasks. During the project, design decisions were refined as implementation details became clearer. This led to a final solution that is more concrete and implementation-oriented than the original design in Report 1.

7.2 Collaboration and Contribution Weights

Table 13 shows the contribution weights for each group member.

s240803	s234146	s245203	s245010	s240335
1.0	1.0	1.0	1.0	1.0

Tabel 13: Contribution weights for the group members

8 Contribution Table

Student Name	Student ID	Contribution Weight
[INSERT NAME]	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE]

Tabel 14: Contribution weights for the group members.

9 Appendix

9.1 Git Repository

The Git repository for the project is available at:

[Insert repository URL here]

9.2 Run Instructions

After unpacking the ZIP archive, the project can be imported as a Maven project in Eclipse, IntelliJ IDEA, or Visual Studio Code.

Run application

```
mvn javafx:run
```

Run tests

```
mvn test
```

The system includes at least one developer with the initials `huba`, as required in the project description.

Figurer

1	VS Code <code>test coverage</code> panel at package level after executing all 57 tests. .	21
2	Per-class coverage within <code>com.planner.repository</code>	21
3	Per-class coverage within <code>com.planner.ui</code>	22
4	Per-class coverage within <code>com.planner.service</code>	23
5	Per-class coverage within <code>com.planner.domain</code>	24
6	The layered architecture of the system	26
7	In-memory repositories	28
8	DIP-flow (skal renames er træt)	32

Tabeller

1	Input properties for <code>createProject(...)</code>	4
2	Concrete test cases for <code>createProject(...)</code>	4
3	Input properties for <code>setActivityDetails(...)</code>	5
4	Concrete test cases for <code>setActivityDetails(...)</code>	6
5	Input properties for <code>addDeveloperToActivity(...)</code>	7
6	Concrete test cases for <code>addDeveloperToActivity(...)</code>	8
7	Input properties for <code>registerTime(...)</code>	10
8	Concrete test cases for <code>registerTime(...)</code>	10
9	Input properties for <code>registerAbsence(...)</code>	10
10	Concrete test cases for <code>registerAbsence(...)</code>	11
11	Line coverage per package as reported by JaCoCo via VS Code's <code>test coverage</code> panel.	20
12	Summary of coverage findings per architectural layer.	25
13	Contribution weights for the group members	35
14	Contribution weights for the group members.	36

Litteratur